Layout Generation Algorithm Design

Phase 1 – Prompt 1.3 Deliverable Module 1 – Dynamic Scenario Generation

Team Sentinels | University of Moratuwa | April 2026
VR-Based Hostage-Rescue Training System | Unity 2022.3 LTS | Meta Quest 3

---

1. Overview

This document defines the complete layout generation algorithm for Module 1's indoor environment generator. The algorithm accepts evaluator-defined parameters from ScenarioConfig.json and produces the `layout` section of Scenario.json — a room graph with corridor connectivity, door placements, entry points, and spatial metadata.

Scope alignment: All generation is single-floor (consistent with ScenarioConfig v1.0.0). Multi-floor is reserved for future extension.

Literature-backed design decisions:

- **Rooms as graph nodes, doors as attributes** — NOT independent entities. [20] demonstrated that treating doors as separate design elements significantly increases generation difficulty.
- **Constructive approach with constraint satisfaction** — NOT vanilla WFC. [15] showed WFC cannot enforce global reachability without excessive backtracking.
- **Two-stage design** — Generate layout structure first, validate reachability separately. [17]'s sequential generation flow directly supports this.
- **Single scenario per request** — NOT exhaustive enumeration. [19]'s scalability challenges (memory exhaustion on large examples) inform this.

---

2. Data Structures

2.1 RoomNode

```
RoomNode {
    id:                 string          // "room_01", "room_02", ...
    type:               RoomType        // entry | corridor | standard |
hostage_room
    position:           Vector3         // centre in Unity world coordinates (y=0)
    size:               RoomSize        // { width, depth, height } in metres
```

```
    connectedRoomIds:  List<string>    // IDs of rooms connected via doors
    doors:             List<DoorData>  // doors as attributes of THIS room
    depth:             int             // BFS depth from entry (set after graph
construction)
    zoneLabel:         string          // human-readable label (set after graph
construction)
}
```

2.2 DoorData

```
DoorData {
    id:                string          // "door_01_02"
    connectsToRoomId:  string          // room on the other side
    position:          Vector3         // world position on shared wall
    wallSide:          WallSide        // north | south | east | west
}
```

2.3 Layout (Output)

```
Layout {
    rooms:             List<RoomNode>
    entryPoints:       List<EntryPoint>
    layoutMetadata:    LayoutMetadata
}

EntryPoint {
    id:                string          // "entry_main"
    roomId:            string
    position:          Vector3
    facingDirection:   Vector3
}

LayoutMetadata {
    totalRooms:        int
    layoutType:        string
    maxDepth:          int
    graphDiameter:     int
    roomSizeCategory:  string
    entryType:         string
    boundingBox:       { min: Vector3, max: Vector3 }
}
```

2.4 Room Size Mapping

Category	Width (m)	Depth (m)	Height (m)	Corridor Gap (m)
small	4.0	4.0	3.0	2.0
medium	6.0	6.0	3.0	2.0
large	8.0	8.0	3.0	2.0

The corridor gap is the spacing between adjacent rooms where a connecting corridor/doorway is implied.

---

3. High-Level Algorithm: LayoutGenerator.Generate()

```
FUNCTION Generate(config: ScenarioConfig, rng: System.Random) -> Layout

    // ── STAGE 1: Determine Parameters ──
    roomSize ← LookupRoomSize(config.missionStructure.roomSize)
    roomCount ← SelectRoomCount(config.missionStructure.roomCount,
                                config.executionControls.randomnessLevel, rng)
    layoutType ← config.missionStructure.layoutType
    entryType ← config.missionStructure.entryType

    // ── STAGE 2: Generate Graph Topology ──
    // Constructive approach: build valid topology directly [15][17]
    adjacencyList ← SWITCH layoutType:
        "linear"          → GenerateLinearTopology(roomCount, rng)
        "branching"       → GenerateBranchingTopology(roomCount, rng)
        "hub_and_spoke"   → GenerateHubAndSpokeTopology(roomCount, rng)
        "loop"            → GenerateLoopTopology(roomCount, rng)

    // ── STAGE 3: Assign Spatial Positions ──
    rooms ← AssignRoomPositions(adjacencyList, roomSize, rng,
                                config.executionControls.randomnessLevel)

    // ── STAGE 4: Place Doors on Shared Walls ──
    // Doors are attributes of rooms, not independent entities [20]
    PlaceDoors(rooms, adjacencyList, rng)

    // ── STAGE 5: Assign Room Metadata ──
```

```
    AssignRoomDepths(rooms, entryRoomId)       // BFS from entry
    AssignRoomTypes(rooms)                      // entry, standard, etc.
    AssignZoneLabels(rooms)                     // human-readable labels

    // ── STAGE 6: Create Entry Points ──
    entryPoints ← CreateEntryPoints(rooms, entryType, rng)

    // ── STAGE 7: Compute Layout Metadata ──
    metadata ← ComputeLayoutMetadata(rooms, layoutType, roomSize, entryType)

    // ── STAGE 8: Validate Reachability (Stage 2 of two-stage design [17]) ──
    IF NOT ValidateReachability(rooms, entryPoints[0].roomId) THEN
        THROW GenerationException("Unreachable rooms detected")

    RETURN Layout { rooms, entryPoints, metadata }
END FUNCTION
```

3.1 SelectRoomCount

```
FUNCTION SelectRoomCount(range: {min, max}, randomnessLevel: string, rng) -> int
    SWITCH randomnessLevel:
        "low"    → RETURN range.min    // deterministic: always use minimum
        "medium" → RETURN rng.Next(range.min, range.max + 1)  // uniform random
        "high"   → RETURN rng.Next(range.min, range.max + 1)  // uniform random
                   // (high randomness affects topology variation, not count)
END FUNCTION
```

---

4. Topology Generation: Per-Layout-Type Pseudocode

4.1 Linear Topology

Structure: A sequential chain of rooms. Each room connects to at most two neighbours (previous and next). The entry is at one end, creating a single path with no decision points.

Tactical character: Forces sequential room clearing. Maximum depth equals room count minus one. Predictable flow — suitable for lower difficulty or training exercises focused on methodical clearing.

```
FUNCTION GenerateLinearTopology(roomCount: int, rng) -> AdjacencyList
    adj ← new AdjacencyList()

    FOR i ← 0 TO roomCount - 1:
        roomId ← "room_" + PadZero(i + 1)
        adj.AddNode(roomId)
        IF i > 0 THEN
            prevId ← "room_" + PadZero(i)
            adj.AddEdge(prevId, roomId)    // chain link

    entryRoomId ← "room_01"
    adj.SetEntry(entryRoomId)
    RETURN adj
END FUNCTION
```

Graph shape (5 rooms):

```
[room_01] ── [room_02] ── [room_03] ── [room_04] ── [room_05]
  (entry)                                              (deepest)
```

4.2 Branching Topology

Structure: A tree with one or more branch points. The entry is the root. Each non-leaf node can have 2–3 children. Creates decision points where the trainee must choose which branch to clear first.

Tactical character: Introduces uncertainty — the trainee cannot know which branch contains the hostage. Higher room counts allow more branch points. Depth varies by branch.

```
FUNCTION GenerateBranchingTopology(roomCount: int, rng) -> AdjacencyList
    adj ← new AdjacencyList()
    entryId ← "room_01"
    adj.AddNode(entryId)
    adj.SetEntry(entryId)

    // Build tree using BFS-style expansion
    frontier ← Queue containing entryId
    roomsPlaced ← 1
    nextId ← 2

    WHILE roomsPlaced < roomCount AND frontier is not empty:
        parentId ← frontier.Dequeue()

        // Decide number of children: 1–3 (capped by remaining rooms)
        maxChildren ← MIN(3, roomCount - roomsPlaced)
        IF maxChildren <= 0 THEN BREAK

        // At least 1 child for the first node to ensure connectivity
        // Random 1–maxChildren for branching variation
        numChildren ← rng.Next(1, maxChildren + 1)

        // Ensure at least one branch point exists (first node gets 2+ if
possible)
        IF parentId == entryId AND roomCount >= 4 AND numChildren < 2 THEN
            numChildren ← 2

        FOR c ← 0 TO numChildren - 1:
            IF roomsPlaced >= roomCount THEN BREAK
            childId ← "room_" + PadZero(nextId)
            adj.AddNode(childId)
            adj.AddEdge(parentId, childId)
            frontier.Enqueue(childId)
            nextId ← nextId + 1
            roomsPlaced ← roomsPlaced + 1

    RETURN adj
END FUNCTION
```

Graph shape (5 rooms, one branch point):

```
            [room_03] ── [room_05]
           /
[room_01] ── [room_02]
  (entry)    \
              [room_04]
```

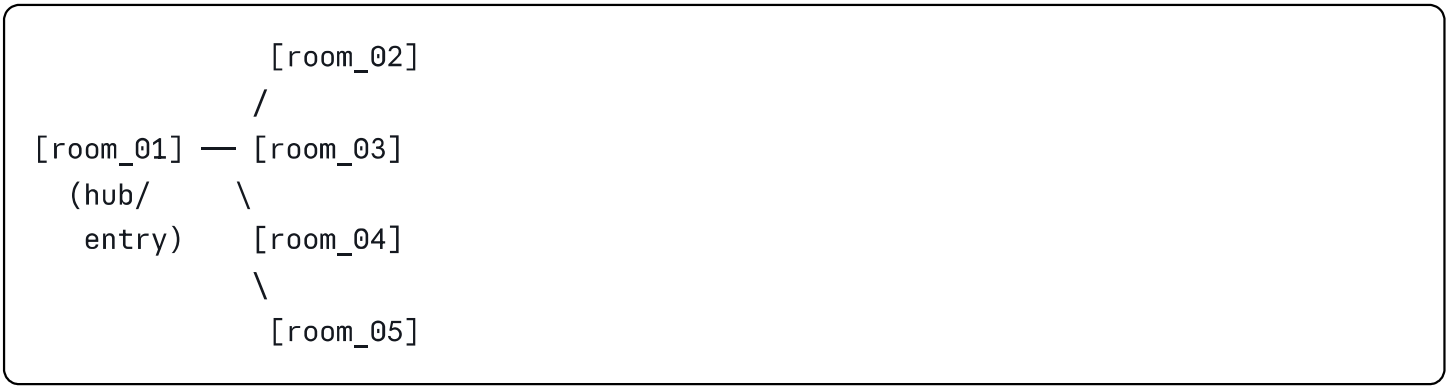
4.3 Hub-and-Spoke Topology

Structure: One central hub room connected to all other rooms as spokes. Creates a star topology where the hub is a critical chokepoint. All paths go through the hub.

Tactical character: The hub room is the highest-connectivity node — natural location for patrol terrorists. Spoke rooms are all depth-1 dead ends — natural guard positions. Forces the trainee to clear or control the hub before reaching any spoke.

```
FUNCTION GenerateHubAndSpokeTopology(roomCount: int, rng) -> AdjacencyList
    adj ← new AdjacencyList()

    // The hub is room_01 (also the entry room)
    hubId ← "room_01"
    adj.AddNode(hubId)
    adj.SetEntry(hubId)

    // Remaining rooms are spokes
    FOR i ← 2 TO roomCount:
        spokeId ← "room_" + PadZero(i)
        adj.AddNode(spokeId)
        adj.AddEdge(hubId, spokeId)

    // Optional: for roomCount >= 6, extend one spoke into a chain of 2
    // to add depth variation (controlled by randomness)
    // This creates one "deep" path among shallow spokes
    IF roomCount >= 6 AND rng.NextDouble() < 0.5 THEN
        // Pick a random spoke and extend it
        extendSpoke ← "room_" + PadZero(rng.Next(2, roomCount + 1))
        // Actually: this would change roomCount, so skip for core implementation
        // Depth variation is handled post-generation in entity placement
        PASS

    RETURN adj
END FUNCTION
```

Graph shape (5 rooms):



4.4 Loop Topology

Structure: A ring of rooms forming a cycle, optionally with interior cross-connections. Creates a layout where the trainee can approach rooms from two directions — enabling flanking.

Tactical character: No dead ends in the core ring. Every room has at least two exits. The trainee can choose clockwise or counterclockwise approach. Highest tactical complexity of the four types.

```
FUNCTION GenerateLoopTopology(roomCount: int, rng) -> AdjacencyList
    adj ← new AdjacencyList()

    // Minimum 4 rooms for a meaningful loop
    effectiveCount ← MAX(roomCount, 4)

    // Create ring: room_01 → room_02 → ... → room_N → room_01
    FOR i ← 1 TO effectiveCount:
        roomId ← "room_" + PadZero(i)
        adj.AddNode(roomId)

    FOR i ← 1 TO effectiveCount:
        currentId ← "room_" + PadZero(i)
        nextId ← "room_" + PadZero((i % effectiveCount) + 1)
        adj.AddEdge(currentId, nextId)

    adj.SetEntry("room_01")

    // Optional cross-connection for larger loops (6+ rooms)
    // Adds one shortcut across the ring
    IF effectiveCount >= 6 AND rng.NextDouble() < 0.4 THEN
        // Connect two rooms roughly opposite in the ring
        a ← 1
        b ← (effectiveCount / 2) + 1
        aId ← "room_" + PadZero(a)
        bId ← "room_" + PadZero(b)
        IF NOT adj.HasEdge(aId, bId) THEN
            adj.AddEdge(aId, bId)

    RETURN adj
END FUNCTION
```

Graph shape (5 rooms):

```
[room_01] ── [room_02]
  (entry) \           \
           \     [room_03]
            \         /
[room_05] ── [room_04]
```

---

5. Spatial Position Assignment

```
FUNCTION AssignRoomPositions(adj: AdjacencyList, roomSize: RoomSize,
                             rng, randomnessLevel: string) -> List<RoomNode>
    rooms ← empty list
    placed ← empty dictionary (roomId → Vector3)
    gap ← 2.0   // corridor gap between rooms

    // BFS from entry to place rooms outward
    entryId ← adj.GetEntry()
    placed[entryId] ← Vector3(0, 0, 0)
    queue ← Queue containing entryId
    visited ← Set containing entryId

    // Direction options for placing children relative to parent
    directions ← [ Vector3(1,0,0),   // east
                   Vector3(0,0,1),   // north
                   Vector3(-1,0,0),  // west
                   Vector3(0,0,-1) ] // south

    WHILE queue is not empty:
        parentId ← queue.Dequeue()
        parentPos ← placed[parentId]

        neighbours ← adj.GetNeighbours(parentId) filtered to unvisited

        // Shuffle available directions for variation
        availDirs ← ShuffleDirections(directions, rng, randomnessLevel)
        dirIndex ← 0

        FOR EACH neighbourId IN neighbours:
            IF neighbourId IN visited THEN CONTINUE

            // Pick next available direction
            dir ← availDirs[dirIndex % availDirs.Length]
            dirIndex ← dirIndex + 1

            // Calculate position: parent + direction * (roomWidth + gap)
            offset ← dir * (roomSize.width + gap)
            candidatePos ← parentPos + offset

            // Check for overlap with already-placed rooms
            attempts ← 0
```

```
            WHILE OverlapsExisting(candidatePos, roomSize, placed) AND attempts <
8:
                dirIndex ← dirIndex + 1
                dir ← availDirs[dirIndex % availDirs.Length]
                offset ← dir * (roomSize.width + gap)
                candidatePos ← parentPos + offset
                attempts ← attempts + 1

            // Apply randomness jitter
            IF randomnessLevel == "high" THEN
                jitter ← rng.NextDouble() * 1.0 - 0.5  // ±0.5m
                candidatePos.x ← candidatePos.x + jitter
                candidatePos.z ← candidatePos.z + jitter

            placed[neighbourId] ← candidatePos
            visited.Add(neighbourId)
            queue.Enqueue(neighbourId)

    // Build RoomNode list
    FOR EACH (roomId, pos) IN placed:
        room ← new RoomNode {
            id = roomId,
            position = pos,
            size = roomSize,
            connectedRoomIds = adj.GetNeighbours(roomId),
            doors = [],     // populated in PlaceDoors
            depth = -1,     // populated in AssignRoomDepths
            zoneLabel = ""  // populated in AssignZoneLabels
        }
        rooms.Add(room)

    RETURN rooms
END FUNCTION
```

5.1 ShuffleDirections (Randomness-Controlled)

```
FUNCTION ShuffleDirections(dirs: Vector3[], rng, randomnessLevel: string) ->
Vector3[]
    result ← copy of dirs
    SWITCH randomnessLevel:
        "low"    → RETURN result   // no shuffle — deterministic placement order
        "medium" → FisherYatesShuffle(result, rng)  // random order
        "high"   → FisherYatesShuffle(result, rng)  // random order + jitter
```

```
applied separately
    RETURN result
END FUNCTION
```

---

6. Door Placement

Doors are placed as attributes of rooms on shared walls. For each edge in the adjacency list, we determine which wall is shared between the two rooms and place a door on that wall.

```
FUNCTION PlaceDoors(rooms: List<RoomNode>, adj: AdjacencyList, rng)
    processedEdges ← empty Set of (string, string) pairs

    FOR EACH room IN rooms:
        FOR EACH neighbourId IN room.connectedRoomIds:
            edgeKey ← SortedPair(room.id, neighbourId)
            IF edgeKey IN processedEdges THEN CONTINUE
            processedEdges.Add(edgeKey)

            neighbour ← FindRoom(rooms, neighbourId)

            // Determine shared wall based on relative positions
            delta ← neighbour.position - room.position
            wallSide ← DetermineWallSide(delta)
            oppositeWall ← OppositeWall(wallSide)

            // Door position: midpoint of shared wall
            doorPos ← CalculateDoorPosition(room, neighbour, wallSide)

            // Create door ID
            doorId ← "door_" + room.id.suffix + "_" + neighbourId.suffix

            // Add door to THIS room
            room.doors.Add(new DoorData {
                id = doorId,
                connectsToRoomId = neighbourId,
                position = doorPos,
                wallSide = wallSide
            })

            // Add reciprocal door to neighbour
            neighbour.doors.Add(new DoorData {
                id = doorId,
                connectsToRoomId = room.id,
                position = doorPos,
                wallSide = oppositeWall
            })

END FUNCTION

FUNCTION DetermineWallSide(delta: Vector3) -> WallSide
    IF |delta.x| > |delta.z| THEN
        RETURN delta.x > 0 ? "east" : "west"
```

```
    ELSE
        RETURN delta.z > 0 ? "north" : "south"
END FUNCTION
```

---

7. Room Metadata Assignment

7.1 BFS Depth Assignment

```
FUNCTION AssignRoomDepths(rooms: List<RoomNode>, entryRoomId: string)
    entry ← FindRoom(rooms, entryRoomId)
    entry.depth ← 0
    queue ← Queue containing entry
    visited ← Set containing entryRoomId

    WHILE queue is not empty:
        current ← queue.Dequeue()
        FOR EACH neighbourId IN current.connectedRoomIds:
            IF neighbourId NOT IN visited THEN
                neighbour ← FindRoom(rooms, neighbourId)
                neighbour.depth ← current.depth + 1
                visited.Add(neighbourId)
                queue.Enqueue(neighbour)
END FUNCTION
```

7.2 Room Type Assignment

```
FUNCTION AssignRoomTypes(rooms: List<RoomNode>)
    maxDepth ← MAX(room.depth FOR room IN rooms)

    FOR EACH room IN rooms:
        IF room.depth == 0 THEN
            room.type ← "entry"
        ELSE IF room.depth == maxDepth AND room.connectedRoomIds.Count == 1 THEN
            room.type ← "hostage_room"   // deepest dead-end = best hostage
placement
        ELSE
            room.type ← "standard"
END FUNCTION
```

7.3 Zone Label Assignment

```
FUNCTION AssignZoneLabels(rooms: List<RoomNode>)
    labelPrefixes ← ["Entry Hall", "Front Room", "Side Room", "Inner Room",
                     "Back Room", "Storage Room", "Office", "Utility Room"]
    usedLabels ← empty set

    FOR EACH room IN rooms (sorted by depth, then by id):
        IF room.type == "entry" THEN
            room.zoneLabel ← "Entry Hall"
        ELSE IF room.type == "hostage_room" THEN
            room.zoneLabel ← "Back Office"
        ELSE
            // Assign from prefix list, ensuring uniqueness
            label ← SelectUniqueLabel(labelPrefixes, room.depth, usedLabels)
            room.zoneLabel ← label
        usedLabels.Add(room.zoneLabel)
END FUNCTION
```

---

8. How Randomness Level Affects Each Generation Step

Generation Step	Low (Deterministic)	Medium (Moderate)	High (Maximum Variation)
Room count selection	Always use `roomCount.min`	Random within [min, max]	Random within [min, max]
Topology construction	Deterministic branch order (first child = primary path)	Random child count per node (1–3)	Random child count + random branch order
Direction shuffling	Fixed order: east → north → west → south	Fisher-Yates shuffle of directions	Fisher-Yates shuffle of directions
Position jitter	None (rooms on exact grid)	None	±0.5m random jitter on x and z
Loop cross-connections	Never added	40% chance if 6+ rooms	40% chance if 6+ rooms

Generation Step	Low (Deterministic)	Medium (Moderate)	High (Maximum Variation)
Overall effect	Near-identical layouts from same config (variability comes only from seed)	Moderate structural variation	Maximum structural variation; rooms may be positioned off-grid

Key principle: Low randomness should produce very similar layouts across different seeds (minimal variation). Medium is the default balance. High maximises diversity for RQ4 evaluation (justified by [8]'s novelty search findings and [13]'s scale ablation evidence).

---

9. Entry Point Creation

```
FUNCTION CreateEntryPoints(rooms: List<RoomNode>, entryType: string, rng) ->
List<EntryPoint>
    entryRoom ← FindRoom(rooms, room where depth == 0)
    entryPoints ← []

    // Primary entry: positioned just outside the entry room
    primaryPos ← entryRoom.position - Vector3(entryRoom.size.width/2 + 1.0, 0, 0)
    entryPoints.Add(new EntryPoint {
        id = "entry_main",
        roomId = entryRoom.id,
        position = primaryPos,
        facingDirection = Vector3(1, 0, 0)  // facing into the room
    })

    IF entryType == "multiple" THEN
        // Find 1-2 additional rooms at depth 0 or 1 that have exterior walls
        candidates ← rooms WHERE depth <= 1 AND id != entryRoom.id
        secondaryCount ← MIN(candidates.Count, 2)

        FOR i ← 0 TO secondaryCount - 1:
            room ← candidates[i]
            pos ← CalculateExteriorEntryPosition(room)
            entryPoints.Add(new EntryPoint {
                id = "entry_secondary_" + (i + 1),
                roomId = room.id,
                position = pos,
                facingDirection = CalculateFacingInto(room, pos)
            })
```

```
    RETURN entryPoints
END FUNCTION
```

---

10. Reachability Validation (Two-Stage Design, Stage 2)

Following [17]'s recommendation for separate validation after constructive generation:

```
FUNCTION ValidateReachability(rooms: List<RoomNode>, entryRoomId: string) -> bool
    visited ← BFS(rooms, entryRoomId)
    RETURN visited.Count == rooms.Count    // all rooms reachable from entry
END FUNCTION
```

This validation is fast (O(V+E)) and catches any construction errors. If it fails, the orchestrator (ScenarioGenerator) retries with a different seed.

---

11. Worked Example: 5-Room Branching Layout

Input parameters:

- `roomCount`: { min: 5, max: 5 }
- `roomSize`: "medium" (6×6m)
- `layoutType`: "branching"
- `entryType`: "single"
- `randomnessLevel`: "medium"
- `seed`: 42

Step 1: Room Count Selection

- randomnessLevel = "medium" → rng.Next(5, 6) = 5

Step 2: Branching Topology Generation

- Create root: room_01 (entry)
- Frontier: [room_01]
- Process room_01: numChildren = 2 (entry gets ≥2 branches)
    - Add room_02, edge (room_01 → room_02)

    - Add room_03, edge (room_01 → room_03)
    - Frontier: [room_02, room_03], placed = 3
- Process room_02: numChildren = rng.Next(1,3) = 1
    - Add room_04, edge (room_02 → room_04)
    - Frontier: [room_03, room_04], placed = 4
- Process room_03: numChildren = rng.Next(1,2) = 1
    - Add room_05, edge (room_03 → room_05)
    - Frontier: [room_04, room_05], placed = 5 ✓ done

Adjacency list:

```
room_01 → [room_02, room_03]
room_02 → [room_01, room_04]
room_03 → [room_01, room_05]
room_04 → [room_02]
room_05 → [room_03]
```

Step 3: Spatial Position Assignment (medium size = 6m, gap = 2m, stride = 8m)

- room_01: (0, 0, 0) — entry
- Directions shuffled (seed=42): [east, north, west, south]
- room_02 placed east of room_01: (8, 0, 0)
- room_03 placed north of room_01: (0, 0, 8)
- room_04 placed east of room_02: (16, 0, 0)
- room_05 placed north of room_03: (0, 0, 16)

Step 4: Door Placement

Door ID	Room A	Room B	Wall Side (A)	Position
door_01_02	room_01	room_02	east	(4, 0, 0)
door_01_03	room_01	room_03	north	(0, 0, 4)
door_02_04	room_02	room_04	east	(12, 0, 0)
door_03_05	room_03	room_05	north	(0, 0, 12)

Step 5: BFS Depth Assignment (from room_01)

Room	Depth
room_01	0
room_02	1
room_03	1
room_04	2
room_05	2

Step 6: Room Type & Zone Label Assignment

Room	Depth	Connections	Type	Zone Label
room_01	0	2	entry	Entry Hall
room_02	1	2	standard	East Corridor
room_03	1	2	standard	North Wing
room_04	2	1	hostage_room	Back Office
room_05	2	1	standard	North Dead End

Step 7: Entry Point

```json
{
  "id": "entry_main",
  "roomId": "room_01",
  "position": { "x": -4.0, "y": 0.0, "z": 0.0 },
  "facingDirection": { "x": 1.0, "y": 0.0, "z": 0.0 }
}
```

Step 8: Validation

- BFS from room_01 visits: room_01 → room_02, room_03 → room_04, room_05
- Visited count = 5 = total rooms ✓ All rooms reachable

Visual Layout

```
                    (0, 0, 16)
                    [room_05]
                    North Dead End
                    depth=2
                        |
                    (0, 0, 8)
                    [room_03]
                    North Wing
                    depth=1
                        |
 entry → (0, 0, 0)          (8, 0, 0)          (16, 0, 0)
         [room_01] ———————— [room_02] ———————— [room_04]
         Entry Hall         East Corridor       Back Office
         depth=0            depth=1             depth=2 (hostage_room)
```

Layout Metadata

json

```
{
  "totalRooms": 5,
  "layoutType": "branching",
  "maxDepth": 2,
  "graphDiameter": 4,
  "roomSizeCategory": "medium",
  "entryType": "single",
  "boundingBox": {
    "min": { "x": -4.0, "y": 0.0, "z": -3.0 },
    "max": { "x": 19.0, "y": 3.0, "z": 19.0 }
  }
}
```

---

12. Seed-Based Reproducibility

All randomised decisions flow through the single `System.Random` instance initialised from the config seed. The generation is fully deterministic for a given seed:

1. `System.Random rng = new System.Random(config.executionControls.seed ?? Environment.TickCount)`

2. The auto-generated seed (when null) is captured and stored in `configurationMetadata.seedUsed`
3. All calls to `rng.Next()` and `rng.NextDouble()` occur in a deterministic order within the pipeline
4. Same seed + same ScenarioConfig = identical Layout every time

This satisfies the reproducibility requirement for RQ4 evaluation and [22]'s traceability requirement.

---

13. Design Decisions Summary with Literature References

Decision	Justification
Rooms as graph nodes, doors as room attributes	[20] showed this reduces generation difficulty vs. independent door entities
Constructive topology generation (not WFC)	[15] proved WFC cannot enforce global reachability without excessive backtracking
Two-stage: generate then validate	[17]'s sequential flow: functional structure first, then constraint validation
Single scenario per request	[19]'s scalability findings: exhaustive enumeration causes memory exhaustion
BFS depth for room metadata	[18]'s path DAG / resistor network analog for flow direction
Room adjacency graph as representation	[14]'s probabilistic grammar over room types provides precedent
Space Syntax-style type assignment	[11]'s accessibility graph: deep nodes = private/guarded, shallow = public/patrol
Randomness controlling structural variation	[8]'s novelty search + [13]'s scale ablation evidence for diversity

---

End of Layout Generation Algorithm Design | Phase 1, Prompt 1.3 | Module 1 – Dynamic Scenario Generation